

6장 프로세스

숙명여대 창병모

6장 프로세스

- 01 프로세스
- 02 작업 제어
- 03 프로세스 제어
- 04 프로세스와 사용자 ID
- 05 시그널과 프로세스



6.1 프로세스

프로세스(process)

- 실행중인 프로그램을 **프로세스**(process)라고 한다.
- 프로세스 번호
 - 각 프로세스는 유일한 프로세스 번호 PID를 갖는다.
- 부모 프로세스
 - 각 프로세스는 부모 프로세스에 의해 생성된다.
- 프로세스의 종류
 - 시스템 프로세스
시스템 운영에 필요한 기능을 수행하는 프로세스.
어떤 서비스를 위해 부팅 시 생성되는 데몬 프로세스 대표적인 예.
 - 사용자 프로세스
사용자의 명령 혹은 프로그램을 실행시키기 위해 생성된 프로세스.

프로세스 상태 보기: **ps**(process status)

- 사용법

```
$ ps [-옵션]
```

현재 시스템 내에 존재하는 프로세스들의 실행 상태를 요약해서 출력한다.

- 사용 예

```
$ ps
```

PID	TTY	TIME	CMD
1519	pts/3	00:00:00	bash
1551	pts/3	00:00:00	ps

```
$ ps -f
```

UID	PID	PPID	C	STIME	TTY	TIME	CMD
chang	1519	1518	0	17:40	pts/3	00:00:00	/bin/bash
chang	1551	1519	0	17:40	pts/3	00:00:00	ps -f

ps -ef

```
$ ps -ef | more
```

UID	PID	PPID	C	STIME	TTY	TIME	CMD
root	1	0	0	9월30	?	00:00:23	/sbin/init auto noprompt
root	2	0	0	9월30	?	00:00:00	[kthreadd]
root	3	2	0	9월30	?	00:00:00	[rcu_gp]
root	4	2	0	9월30	?	00:00:00	[rcu_par_gp]
root	6	2	0	9월30	?	00:00:00	[kworker/0:0H-events_highpri]
root	9	2	0	9월30	?	00:00:00	[mm_percpu_wq]
root	10	2	0	9월30	?	00:00:00	[rcu_tasks_rude_]
root	11	2	0	9월30	?	00:00:00	[rcu_tasks_trace]
root	12	2	0	9월30	?	00:00:03	[ksoftirqd/0]
root	13	2	0	9월30	?	00:01:07	[rcu_sched]
root	14	2	0	9월30	?	00:00:02	[migration/0]
...							
root	24	2	0	9월30	?	00:00:00	[netns]
root	25	2	0	9월30	?	00:00:00	[inet_frag_wq]
root	26	2	0	9월30	?	00:00:00	[kauditd]

--More--

ps 출력 정보

항목	의미
UID	프로세스를 실행시킨 사용자 ID
PID	프로세스 번호
PPID	부모 프로세스 번호
C	프로세스의 우선순위의
STIME	프로세스의 시작 시간
TTY	명령어가 시작된 터미널
TIME	프로세스에 사용된 CPU 시간
CMD	실행되고 있는 명령어(프로그램) 이름

특정 프로세스 리스트: pgrep

- 특정 프로세스만 리스트

```
$ ps -ef | grep -w sshd
```

- 사용법

```
$ pgrep [옵션] [패턴]
```

패턴에 해당하는 프로세스들만을 리스트 한다.

-l : PID와 함께 프로세스의 이름을 출력한다.

-f : 명령어의 경로도 출력한다.

-n : 패턴과 일치하는 프로세스들 중에서 가장 최근 프로세스만을 출력한다.

-x : 패턴과 정확하게 일치되는 프로세스만 출력한다.

-a : 전체 명령어 줄과 PID를 출력한다.

특정 프로세스 리스트: pgrep

- 예

```
$ pgrep sshd  
5032
```

- -l 옵션: 프로세스 번호와 프로세스 이름을 함께 출력

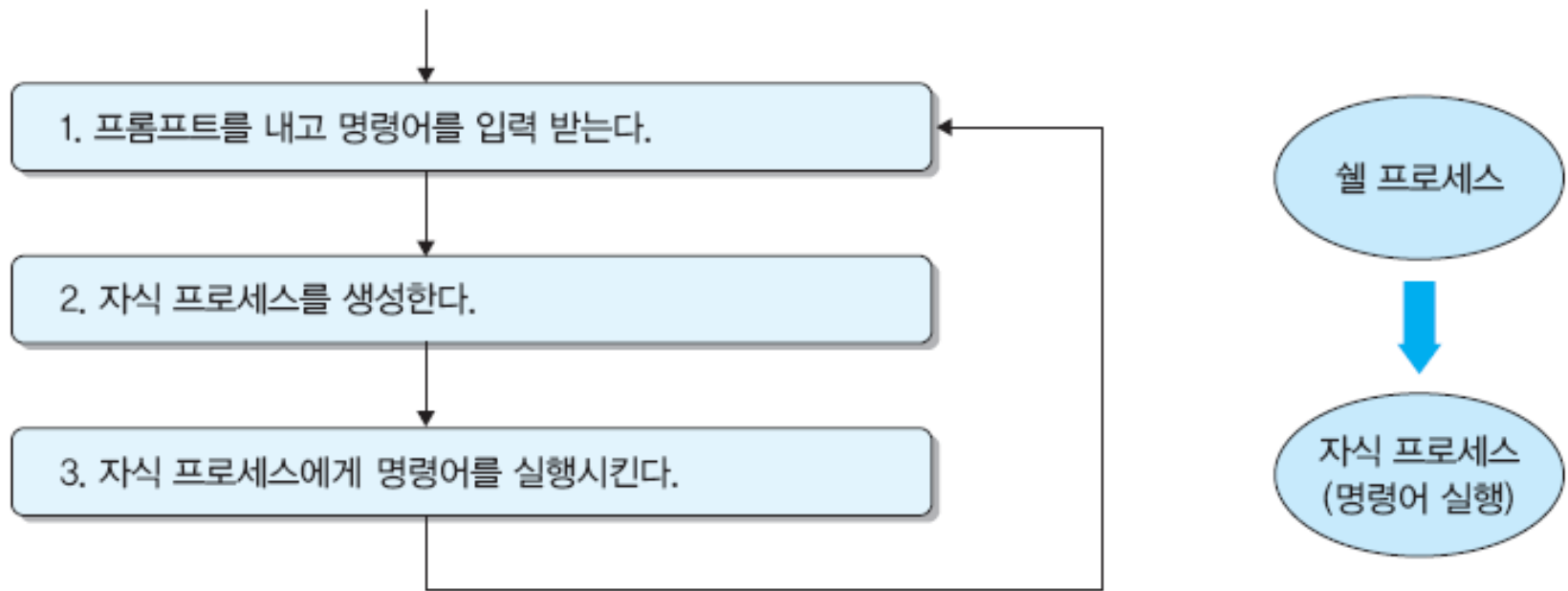
```
$ pgrep -l sshd  
5032 sshd
```

- -n 옵션: 가장 최근 프로세스만 출력한다.

```
$ pgrep -ln sshd  
5032 sshd
```

6.2 작업 제어

셸과 프로세스



후면 처리

\$ 명령어 &
[1] 프로세스번호

- 예
\$ sleep 10 &
[1] 6530
\$ ps
PID TTY TIME CMD
1519 pts/0 00:00:00 bash
6530 pts/0 00:00:00 sleep
6535 pts/0 00:00:00 ps



셸 재우기

- 사용법

```
$ sleep 초
```

명시된 시간만큼 프로세스 실행을 중지시킨다.

- 예

```
$ (echo 시작; sleep 5; echo 끝)
```

강제 종료

- 강제종료 Ctrl-C

\$ 명령어

^C

- 예

\$ (sleep 100; echo DONE)

^C

\$

- 실행 정지 Ctrl-Z

\$ 명령어

^Z

[1]+ 정지됨 명령어

후면 작업의 전면 전환: fg(foreground)

```
$ fg
```

정지된 작업을 다시 전면에서 실행시킨다.

- 예

```
$ (sleep 100; echo done)
```

```
^Z
```

```
[1]+ 정지됨 ( sleep 100; echo DONE )
```

```
$ fg
```

```
( sleep 100; echo DONE )
```

후면 작업의 전면 전환: fg(foreground)

```
$ fg %작업번호
```

작업번호에 해당하는 후면 작업을 전면 작업으로 전환시킨다.

- 예

```
$ (sleep 100; echo DONE) &
```

```
[1] 10067
```

```
$ fg %1
```

```
( sleep 100; echo DONE )
```


전면 작업의 후면 전환: bg(background)

- 사용법

- Ctrl-Z 키를 눌러 전면 실행중인 작업을 먼저 중지시킨 후
- bg 명령어 사용하여 후면 작업으로 전환

`$ bg %작업번호`

작업번호에 해당하는 중지된 작업을 후면 작업으로 전환하여 실행한다.

- 예

```
$ ( sleep 100; echo DONE )
```

```
^Z
```

```
[1]+  정지됨    ( sleep 100; echo DONE )
```

```
$ bg %1
```

```
[1]+ ( sleep 100; echo DONE ) &
```

후면 작업의 입출력 제어

- 후면 작업의 출력

\$ 명령어 > 출력파일 &

\$ find . -name test.c -print > find.txt &

\$ find . -name test.c -print | mail chang &

- 후면 작업의 입력

\$ 명령어 < 입력파일 &

6.3 프로세스 제어

프로세스 끝내기: kill

- 프로세스 강제 종료

```
$ kill 프로세스번호
```

```
$ kill %작업번호
```

프로세스 번호(혹은 작업 번호)에 해당하는 프로세스를 강제로 종료시킨다.

- 예

```
$ (sleep 100; echo done) &
```

```
[1] 8320
```

```
$ kill 8320 혹은 $ kill %1
```

```
[1] 종료됨    ( sleep 100; echo done )
```

- exit 명령어

```
exit [종료코드]
```

프로세스 기다리기: wait

- 사용법

```
$ wait [프로세스번호]
```

프로세스 번호로 지정한 자식 프로세스가 종료될 때까지 기다린다.

지정하지 않으면 모든 자식 프로세스가 끝나기를 기다린다.

- 예

```
$ (sleep 10; echo 1번 끝) &
```

```
[1] 1231
```

```
$ echo 2번 끝; wait 1231; echo 3번 끝
```

```
2번 끝
```

```
1번 끝
```

```
3번 끝
```

프로세스 기다리기: wait

- 예

```
$ (sleep 10; echo 1번 끝) &  
$ (sleep 10; echo 2번 끝) &  
$ echo 3번 끝; wait; echo 4번 끝  
3번 끝  
1번 끝  
2번 끝  
4번 끝
```

프로세스 우선순위

- 실행 우선순위 nice 값
 - 19(제일 낮음) ~ -20(제일 높음)
 - 보통 기본 우선순위 0으로 명령어를 실행
- nice 명령어

`$ nice [-n 조정수치] 명령어 [인수들]`

주어진 명령을 조정된 우선순위로 실행한다.

- 예
 - `$ nice` // 현재 우선순위 출력
 - `0`
 - `$ nice -n 10 ps -ef` // 조정된 우선순위로 실행

프로세스 우선순위 조정

- 사용법

```
$ renice [-n] 우선순위 [-gpu] PID
```

이미 수행중인 프로세스의 우선순위를 명시된 우선순위로 변경한다.

-g : 해당 그룹명 소유로 된 프로세스를 의미한다.

-u : 지정한 사용자명의 소유로 된 프로세스를 의미한다.

-p : 해당 프로세스의 PID를 지정한다.

6.4 프로세스의 사용자 ID

프로세스의 사용자 ID

- 프로세스는 사용자 ID와 그룹 ID를 갖는다.
 - 그 프로세스를 실행시킨 사용자의 ID와 사용자의 그룹 ID
 - 프로세스가 수행할 수 있는 연산을 결정하는 데 사용된다.
- id 명령어

```
$ id [사용자명]
```

사용자의 실제 ID와 유효 사용자 ID, 그룹 ID 등을 보여준다.

```
$ id
```

```
uid=1000(chang) gid=1000(chang) 그룹들=1000(chang),4(adm),24(cdrom),  
27(sudo),30(dip),46(plugdev),121(lpadmin),132(lxd),133(sambashare)
```

```
$ echo $UID $EUID
```

```
1000 1000
```

프로세스의 사용자 ID

- 프로세스의 **실제 사용자 ID(real user ID)**
 - 그 프로세스를 실행시킨 사용자의 ID로 설정된다.
 - 예: chang 사용자 ID로 로그인하여 어떤 프로그램을 실행시키면 그 프로세스의 실제 사용자 ID는 chang이 된다.
- 프로세스의 **유효 사용자 ID(effective user ID)**
 - 현재 유효한 사용자 ID
 - 보통 유효 사용자 ID와 실제 사용자 ID는 같다.
 - 새로 파일을 만들 때나 파일의 접근권한을 검사할 때 주로 사용됨
 - **특별한 실행파일**을 실행할 때 유효 사용자 ID는 달라진다.

프로세스

실제 사용자 ID

유효 사용자 ID

set-user-id 실행파일

- set-user-id(set user ID upon execution) 실행권한
 - set-user-id가 설정된 실행파일을 실행하면
 - 이 프로세스의 유효 사용자 ID는 그 실행파일의 소유자로 바뀜.
 - 이 프로세스는 실행되는 동안 그 파일의 소유자 권한을 갖게 됨.

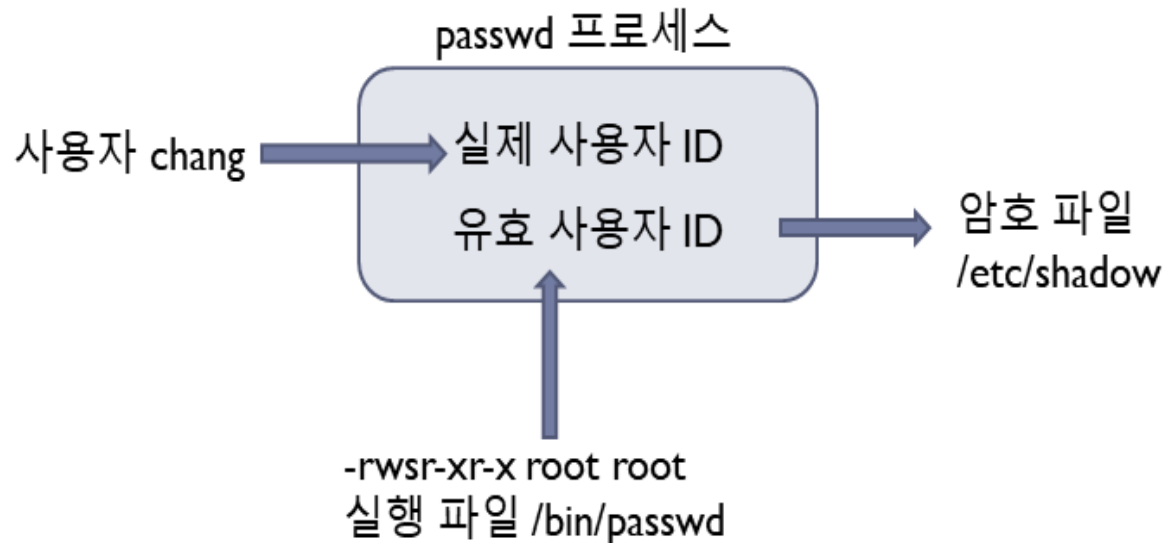
- 예

```
$ ls -l /bin/passwd
```

```
-rwsr-xr-x. 1 root  root  59976  7월 14 14:57 /bin/passwd
```

set-user-id 실행파일을 실행하는 과정

- (1) /bin/passwd 파일은 set-user-id 실행권한이 설정된 실행파일이며 소유자는 root
- (2) 일반 사용자가 이 파일을 실행하면 이 프로세스의 유효 사용자 ID는 root가 됨.
- (3) 유효 사용자 ID가 root이므로 root만 수정할 수 있는 암호 파일 /etc/shadow 파일을 접근하여 수정



set-group-id 실행파일

- set-group-id(set group ID upon execution) 실행권한
 - 실행되는 동안에 그 파일 소유자의 그룹이 프로세스의 유효 그룹 ID가 된다.
 - set-group-id 실행권한은 8진수 모드로는 2000으로 표현된다.
- set-group-id 실행파일 예

```
$ ls -l /bin/wall  
-r-xr-sr-x. 1 root tty 35200  2월 25  2021 /bin/wall
```

set-user-id/set-group-id 설정

- set-user-id 실행권한 설정
\$ chmod 4755 파일 혹은 \$ chmod u+s 파일
- set-group-id 실행권한 설정
\$ chmod 2755 파일 혹은 \$ chmod g+s 파일

6.5 시그널과 프로세스

시그널

- 시그널을 이용하여 프로세스를 제어한다.
 - 시그널은 예기치 않은 사건이 발생할 때 이를 알리는 소프트웨어 인터럽트이다.
- 시그널 발생 예
 - SIGFPE 부동소수점 오류
 - SIGPWR 정전
 - SIGALRM 알람시계 울림
 - SIGCHLD 자식 프로세스 종료
 - SIGINT 키보드로부터 종료 요청 (Ctrl-C)
 - SIGTSTP 키보드로부터 정지 요청 (Ctrl-Z)



주요 시그널

시그널 이름	의미	기본 처리
SIGABRT	abort()에서 발생하는 종료 시그널	종료(코어 덤프)
SIGALRM	자명종 시계 alarm() 울림 때 발생하는 알람 시그널	종료
SIGCHLD	프로세스의 종료 혹은 정지를 부모에게 알리는 시그널	무시
SIGCONT	정지된 프로세스를 계속시키는 시그널	무시
SIGFPE	0으로 나누기와 같은 심각한 산술 오류	종료(코어 덤프)
SIGHUP	연결 끊김	종료
SIGILL	잘못된 하드웨어 명령어 수행	종료(코어 덤프)
SIGIO	비동기화 I/O 이벤트 알림	종료
SIGINT	터미널에서 Ctrl-C 할 때 발생하는 인터럽트 시그널	종료
SIGKILL	잡을 수 없는 프로세스 종료시키는 시그널	종료
SIGPIPE	파이프에 쓰려는데 리더가 없을 때	종료
SIGPIPE	끊어진 파이프	종료

주요 시그널

SIGPWR	전원고장	종료
SIGSEGV	유효하지 않은 메모리 참조	종료(코어 덤프)
SIGSTOP	프로세스 정지 시그널	정지
SIGTSTP	터미널에서 Ctrl-Z 할 때 발생하는 정지 시그널	정지
SIGSYS	유효하지 않은 시스템 호출	종료(코어 덤프)
SIGTERM	잡을 수 있는 프로세스 종료 시그널	종료
SIGTTIN	후면 프로세스가 제어 터미널을 읽기	정지
SIGTTOU	후면 프로세스가 제어 터미널에 쓰기	정지
SIGUSR1	사용자 정의 시그널	종료
SIGUSR2	사용자 정의 시그널	종료

시그널 리스트

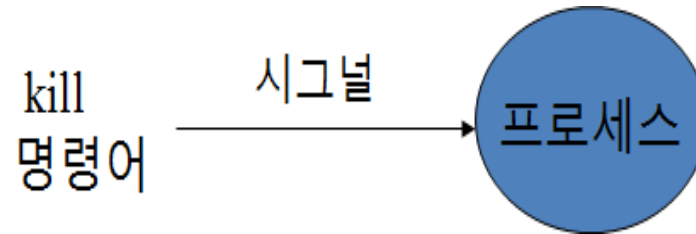
- \$ kill -l

1) SIGHUP	2) SIGINT	3) SIGQUIT	4) SIGILL	5) SIGTRAP
6) SIGABRT	7) SIGBUS	8) SIGFPE	9) SIGKILL	10) SIGUSR1
11) SIGSEGV	12) SIGUSR2	13) SIGPIPE	14) SIGALRM	15) SIGTERM
16) SIGSTKFLT	17) SIGCHLD	18) SIGCONT	19) SIGSTOP	20) SIGTSTP
21) SIGTTIN	22) SIGTTOU	23) SIGURG	24) SIGXCPU	25) SIGXFSZ
26) SIGVTALRM	27) SIGPROF	28) SIGWINCH	29) SIGIO	30) SIGPWR

시그널 보내기: kill 명령어

- kill 명령어

- 한 프로세스가 다른 프로세스를 제어하기 위해 특정 프로세스에 임의의 시그널을 강제로 보낸다.



- 사용법

```
$ kill [-시그널] 프로세스번호
```

```
$ kill [-시그널] %작업번호
```

프로세스 번호(혹은 작업 번호)로 지정된 프로세스에 원하는 시그널을 보낸다.

시그널을 명시하지 않으면 SIGTERM 시그널을 보내 해당 프로세스를 강제 종료

시그널 보내기: kill 명령어

- 종료 시그널 보내기

\$ kill -9 프로세스번호

\$ kill -KILL 프로세스번호

- 다른 시그널 보내기

\$ 명령어 &

[1] 1234

\$ kill -STOP 1234

[1]+ 정지됨 명령어

\$ kill -CONT 1234

핵심 개념

- 프로세스는 실행중인 프로그램이다.
- 각 프로세스는 프로세스 ID를 갖는다. 각 프로세스는 부모 프로세스에 의해 생성된다.
- 셸은 사용자와 운영체제 사이에 창구 역할을 하는 소프트웨어로 사용자로부터 명령어를 입력받아 이를 처리하는 명령어 처리기 역할을 한다.
- 전면 처리는 명령어가 전면에서 실행되므로 셸이 명령어 실행이 끝나기를 기다리지만 후면 처리는 명령어가 후면에서 실행되므로 셸이 명령어 실행이 끝나기를 기다리지 않는다.
- 각 프로세스는 실제 사용자 ID와 유효 사용자 ID를 갖는다.
- 시그널은 예기치 않은 사건이 발생할 때 이를 알리는 소프트웨어 인터럽트이다.
- kill 명령어를 이용하여 특정 프로세스에 원하는 시그널을 보낼 수 있다.